

虛擬電廠即時排程系統

Virtual Power Plant Real-Time Scheduling (VPP-RTS)

即時系統 (Real-Time Systems) 作業

完整實作報告 + 每一評分項的數學式與設計原理

Level 1：日前排程・接收測試・效能評估

Level 2：放寬假設建模・進階動態排程

國立成功大學 即時系統課程

本文件用途與閱讀方式

本報告的目標是：讓每一位組員，對評分表上的每一個得分項，都能說出「它的數學式是什麼」與「為什麼這樣設計」。因此全文採用統一的三段式寫法

- **數學式**：先寫出規格（作業說明 1.3 / 1.4）給定或我們採用的式子。
- **實作對應**：說明程式中如何把該式線性化、寫進 MILP（檔名與限制式名稱）。
- **設計原理**：用一句話講清楚「這條為什麼存在」 它對應到哪個即時系統或電力系統的原理，以及拿掉它會壞掉什麼。

每個小節右側標註 [對應評分 X-Y] 指出它對應評分表的哪一項。文末 §14 另附**備詢速查表**，把所有評分項濃縮成「數學式 + 一句原理」，供報告前最後複習與當場被問時快速翻查。

目錄

本文件用途與閱讀方式	1
第一部分 完整實作說明	2
1 作業背景、符號與目標函數	2
1.1 問題的即時系統對應	2
1.2 符號（與規格 Notation 一致）	3
1.3 目標函數：三個彼此衝突的目標如何被放進「一條式子」	3
2 系統輸入與資產設定	4
2.1 傳統機組（2 部）	4
2.2 再生能源與儲能	4
2.3 市場價格 λ_t	4
3 系統架構與管線	5
4 Phase 1：週期任務集生成（評分 1-1 ~ 1-8）	5
4.1 本次產生的週期任務集（8 個任務，frame size = 4）	5
4.2 逐項說明：每個得分點的數學式與為什麼這樣設計	6
5 Phase 2：日前 MILP 排程 變數、線性化與目標	7
5.1 決策變數	7
5.2 關鍵理論：規格的 $\min(1, \cdot)$ 如何被線性化	7
5.3 目標函數與求解流程	8

目錄	2
6 Phase 2：全部 23 條限制式（規格式 + 實作 + 原理）	8
6.1 任務與供需路由（C1–C3, C5, C20） [對應評分 2-1，5 分]	8
6.2 非週期任務的軟截止（C4） [對應評分 2-2，2 分]	9
6.3 傳統機組（C6–C12） [對應評分 2-3，7 分]	10
6.4 再生能源（C13） [對應評分 2-4，1 分]	10
6.5 儲能（C14–C19, C21） [對應評分 2-5，7 分]	10
6.6 售電與供需平衡（C22, C23） [對應評分 2-6，1 分；2-7，4 分]	11
7 Phase 3：接收測試（評分 4-1 ~ 4-3）	11
7.1 保留量的數學定義（4-1 方法） [對應評分 4-1]	11
7.2 接受/拒絕判斷規則（4-2 合理性） [對應評分 4-2]	12
7.3 Sporadic value rate（4-3） [對應評分 4-3]	12
7.4 本次靜態接收測試結果	12
8 Phase 4：效能評估指標（評分 5-1 ~ 5-5）	13
9 日前保留策略與目標權衡分析（評分 6-1, 6-2）	13
9.1 保留策略：留多少、留在哪、怎麼用（6-1） [對應評分 6-1]	13
9.2 三目標的權衡（6-2） [對應評分 6-2]	14
10 Level 2-A：放寬假設建模 R1–R10（評分 3-1）	14
10.1 放寬參數（本次採用值）	15
10.2 逐條：數學式 + 物理/工程原理	15
11 Level 2-B：進階動態排程（評分 8-1 ~ 8-3）	17
11.1 方法選擇與理由（8-1） [對應評分 8-1]	17
11.2 排程結果正確性（8-2） [對應評分 8-2]	18
12 實際執行結果與靜態 vs 動態比較（評分 8-3）	18
12.1 動態排程的 9 輪再最佳化	19
12.2 靜態（L1）vs 動態（L2）效能比較（8-3） [對應評分 8-3]	19
13 程式碼結構與執行方式	20
14 備詢速查表：評分項 → 數學式 → 一句原理	20
14.1 Level 1（80 分）	21
14.2 Level 2（80~100 分）	22
第二部分 報告講稿	23

第一部分 完整實作說明（數學式 + 設計原理）

1 作業背景、符號與目標函數

1.1 問題的即時系統對應

「虛擬電廠 (Virtual Power Plant, VPP)」把分散的發電與儲能資產整合成一個可被統一調度的電力系統。本作業的核心轉換是：

即時系統概念	VPP 對應
處理器 processor	發電/儲能設備：傳統機組 I_g 、再生能源 I_r 、儲能 I_b
任務 task / 工作 job	有時限的用電需求：週期 J_p 、零星 J_s 、非週期 J_a
工作的執行	在某時段「供應該 job 所需電能」
處理器容量	設備出力上限、SOC、爬升率
排程 schedule	哪個設備在哪個時段供應多少電、賣多少電到市場

時間軸為 $H = 72$ 個離散時段，每格 $\Delta t = 1$ 小時，採時鐘驅動的靜態排程(clock-driven static schedule)：在 $t = 0$ 一次把整個 $[1, 72]$ 的供電表算定。

1.2 符號（與規格 Notation 一致）

符號	意義
$T = \{1, \dots, H\}, \Delta t$	時段集合與時段長度 ($H = 72, \Delta t = 1$)
$I = I_g \cup I_r \cup I_b$	全部設備；機組、再生能源、儲能
$J = J_p \cup J_s \cup J_a, J_{chg}$	全部用電需求； J_{chg} 為「儲能充電需求」（不計入 deadline/response 評估）
J_{np}	非搶占任務集合（執行不可中斷）
r_j, e_j, w_j, d_j, p_j	job 的釋放時間、執行格數、每格能量 (MWh)、相對截止、週期
λ_t	第 t 時段市場售電價 (\$/MWh)
$outputmin_i, outputmax_i, RU_i, RD_i$	機組出力下/上限、爬升率 (up/down)
$UT_i, DT_i, coston_i, costup_i$	最小開/關機時段、固定成本 (\$/h)、變動成本 (\$/MWh)
$TN_i, TF_i, P_{i,0}$	排程前已連續開/關機時段、初始出力
$renewmax_i, renewforecast_{i,t}$	再生能源額定容量、第 t 時段預測出力比例
$stomin_i, stomax_i, dis_i, chg_i, SOC_{i,0}$	儲能 SOC 下/上限、放/充電功率上限、初始 SOC

1.3 目標函數：三個彼此衝突的目標如何被放進「一條式子」

$$\min F = \alpha f_1 + f_2 + f_3 \quad (1)$$

$$f_1 = \sum_{j \in J_a} \text{Miss}_j, \quad \alpha = 10000 \text{ ($/miss)} \text{ (非週期逾時數)} \quad (2)$$

$$f_2 = \sum_{i \in I_g} \sum_{t \in T} \left(\text{coston}_i \cdot \min(1, P_{i,t}) + \text{costup}_i \cdot P_{i,t} \right) \text{ (機組成本)} \quad (3)$$

$$f_3 = - \sum_{t \in T} \lambda_t \text{Sell}_t \text{ (售電收益取負號)} \quad (4)$$

設計原理：這是多目標最佳化的「加權和純量化 (weighted-sum scalarization)」。
三個目標單位不同 (f_1 是「個數」、 f_2, f_3 是「金額」)，無法直接相加，必須先用罰係數 α 把「漏一個非週期任務」折算成金額。

設計原理： $\alpha = 10000$ 的取值刻意遠大於單一任務在不同時段供電的邊際成本差（機組變動成本約 42–70 \$/MWh、單一 job 能量 6–18 MWh，量級在數百到一千元）。因此

最佳化會近似字典序 (lexicographic)：先盡力不漏任何非週期任務，在此前提下才去壓低成本、衝高售電收益。這正是即時系統「先保證時限、再談效率」的精神。 f_3 取負號是把「最大化收益」轉成「最小化負收益」，好讓三項都在同一個 $\min$ 之下。

2 系統輸入與資產設定

輸入檔 `input/processor_settings.json` 與 `input/price_72hr.json` 定義固定資產與價格。本次採用的資產：

2.1 傳統機組 (2 部)

機組	出力下/上限 (MW)	爬升 RU/RD	最小開/關 (h)	固定 (\$/h)	變動 (\$/MWh)
thermal_1	15 / 80	15 / 15	3 / 2	1200	42
thermal_2	10 / 45	20 / 20	2 / 2	600	70

thermal_1 變動成本低 (基載)；*thermal_2* 啟動便宜但每 *MWh* 較貴 (尖峰備用)。兩部初始皆關機。這個「便宜基載 + 昂貴尖峰」結構，是後面成本-收益權衡分析 (§9) 能成立的前提。

2.2 再生能源與儲能

設備	型別	關鍵參數
pv_1	太陽能	額定 60 MW，依日前預測比例出力
pv_2	太陽能	額定 80 MW，預測曲線每日三段日照尖峰 (約 8–13、31–43、55–66 時)
battery_1	儲能	SOC 20–100 MWh，充/放 20 MW，初始 45 MWh
battery_2	儲能	SOC 10–60 MWh，充/放 15 MW，初始 25 MWh

每個電池對應一個充電需求 $j \in J_{chg}$ (*battery_1_chg* / *battery_2_chg*)。充電只能由機組或再生能源供應 (C21)，不能由其他電池放電供應。這對應「不允許電池互充」的物理設定。

2.3 市場價格 λ_t

`price_72hr.json` 提供每小時售電價。本資料尖峰落在第 70 時 (113)、第 22 時 (108)、第 56 時 (105)，谷底為第 12 時 (0)。設計原理：價格隨時段大幅變動，正是排

程要「擇時售電」的動機 多餘電力應留到高價時段賣出，這在 §9 與動態法的權衡裡是關鍵。

3 系統架構與管線

Phase 1	任務生成	generator/	-> output/task_set.json
Phase 2	日前排程	rt_scheduler/	-> output/schedule_result.json (PuLP MILP)
Phase 3	接收測試	acceptance_tester	-> 在排程上標註接受/拒絕 (整合於 Phase 2 之後)
Phase 4	效能評估	evaluator/	-> output/evaluation_results.json
Level 2	動態排程	advanced_scheduler	-> output/*_dynamic.json

設計上採物件導向與單一職責：生成器把計算交給 FrameSizeCalculator、驗證交給 TaskSetValidator、排程器把建模交給 VppMILPFormulator、結果解析交給 SchedulerResultExtractor。資料模型以 Pydantic 定義，I/O 集中於 JsonIO。

4 Phase 1：週期任務集生成（評分 1-1 ~ 1-8）

TaskSetGenerator 以「生成 → 計算 frame size → 驗證」迴圈隨機產生任務，不合格即重抽，直到 TaskSetValidator 全數通過。為保證最難的條件一定成立，生成順序刻意是：先生成 $d = e$ 任務（撐住 1-6）、再生成 $e = 2$ 的非搶占任務（撐住 1-7）、其餘隨機補足。加固定種子（seed）讓任務集可重現。

4.1 本次產生的週期任務集（8 個任務，frame size = 4）

ID	r 釋放	p 週期	e 執行	d 截止	w (MWh)	搶占	job 數
p1	20	20	4	4	11	否	3
p2	1	10	2	8	17	否	7
p3	17	20	4	4	8	否	3
p4	8	8	1	5	13	否	8
p5	8	24	2	19	17	否	2
p6	11	13	2	11	17	否	4
p7	10	16	2	13	16	否	4
p8	9	9	2	7	13	否	7

展開規則：第 k 個 job 的 $\text{release} = r + kp$ 、 $\text{deadline} = \text{release} + d - 1$ ，僅保留 $\text{deadline} \leq 72$ 者；合計 38 個週期 job。

4.2 逐項說明：每個得分點的數學式與為什麼這樣設計

1-1 欄位完整性。 [對應評分 1-1] 每個 task 含 job_id, r, p, e, w, d, preempt 並以 JSON 呈現。**設計原理：**這些是描述一個週期硬即時任務的最小完整參數組 (r, p, e, d) 加上能量需求 w 與可搶占旗標；缺任一個，排程都無法判斷「何時可做、要做多久、要供多少電、可否中斷」。

1-2 任務數 $6 \leq |J_p| \leq 10$ 。 [對應評分 1-2] 本次 $|J_p| = 8$ 。**設計原理：**下限保證問題具有一定規模、上限避免狀態爆炸；落在中間值是合理的測試規模。

1-3 展開 job 數 > 30 。 [對應評分 1-3] 本次 $= 38 > 30$ 。**設計原理：**展開後的 job 數才是排程真正要處理的「工作量」；要求 > 30 確保 72 小時內有足夠密集的硬截止事件，使 23 條限制式真的被觸發、排程不流於空跑。

1-4 參數範圍。 [對應評分 1-4] 需同時滿足： $1 \leq r_j \leq p_j$ ； $6 \leq p_j \leq 24$ 且至少 3 種不同週期； $1 \leq e_j \leq 4$ 、至少 2 個 $e = 2$ 、至少 1 個 $e \geq 3$ ； $e_j \leq d_j \leq p_j$ ； $6 \leq w_j \leq 18$ 、至少 2 個 $w \geq 14$ 。本次：週期 $\{8, 9, 10, 13, 16, 20, 24\}$ 共 7 種； $e = 2$ 有 p2/p5/p6/p7/p8、 $e = 4$ 有 p1/p3； $w \geq 14$ 有 p2/p5/p6/p7。**設計原理：** $e \leq d \leq p$ 保證「一個 job 在下一個同任務 job 釋放前、且在自己截止前」可被做完，避免自我重疊；多種週期與能量讓 hyperperiod 結構與供需曲線有變化，才能檢驗排程的泛用性。

1-5 負載密度 $0.7 \leq D_W = \sum_{j \in J_p} e_j/p_j$ 。 [對應評分

1-5] 本次 $D_W = \frac{4}{20} + \frac{2}{10} + \frac{4}{20} + \frac{1}{8} + \frac{2}{24} + \frac{2}{13} + \frac{2}{16} + \frac{2}{9} = 1.31 \geq 0.7$ 。**設計原理：** D_W 就是即時系統的處理器利用率 (utilization) 類比 每單位時間平均需要多少「執行格」。要求 ≥ 0.7 是確保系統處於中高負載：太低則排程毫無壓力、無法區分演算法好壞。 $D_W > 1$ 在這裡可行，是因為「處理器」不是單一 CPU，而是多部設備可同時供電（規格假設 7：一個 processor 可同時供多個 job），所以總供電能力 > 1 格/時。

1-6 至少 20% 任務 $d_j = e_j$ 。 [對應評分 1-6] 本次 p1、p3 ($2/8 = 25\%$)。**設計原理：** $d = e$ 代表零鬆弛 (zero laxity) 的工作：一釋放就得連續做到截止、沒有任何等待餘地。刻意要求一定比例的零鬆弛任務，是要逼排程器處理「最緊迫、最沒有挪移空間」的硬截止 這是即時排程最吃緊的情況。

1-7 至少 2 個 $e \neq 1$ 的非搶占任務。 [對應評分 1-7] 本次 p2/p5/p6/p7/p8 皆為非搶占且 $e = 2$ 。**設計原理：**若 $e = 1$ ，非搶占與否毫無差別（一格本來就連續）；唯有 $e \geq 2$ 的非搶占任務才會真正觸發「連續區塊」限制 C5，產生阻塞 (blocking) 效應。這條保證 C5 不是擺設。

1-8 Frame size 三條件。 [對應評分 1-8] 取最小可行 f ，需滿足

$$f \geq \max_j e_j, \quad H \bmod f = 0, \quad 2f - \gcd(f, p_j) \leq d_j \quad \forall j. \quad (5)$$

本次 $f = 4$ ： $\max e_j = 4 \checkmark$ ； $72 \bmod 4 = 0 \checkmark$ ；最緊的是 p8： $2 \cdot 4 - \gcd(4, 9) = 8 - 1 = 7 \leq d_{p8} = 7 \checkmark$ 。**設計原理：**這是 Liu 課本時鐘驅動 frame-based 排程的三條經典條件，缺一不可：

- $f \geq \max e_j$ ：一個 frame 必須裝得下最長的一段執行，否則某 job 連在單一 frame 內都跑不完。
- $H \bmod f = 0$ ：整個 horizon 是 frame 的整數倍，cyclic schedule table 才能整齊重複、不會在尾端切出半個 frame。
- $2f - \gcd(f, p_j) \leq d_j$ ：保證在每個 job 的 $[r_j, r_j + d_j - 1]$ 內，至少完整存在一個 frame。 $\gcd(f, p_j)$ 是 release 相對 frame 邊界的最差對齊偏移， $2f - \gcd$ 是「從 release 到下一個可用 frame 結束」的最壞跨距；它 $\leq d_j$ 才能保證 job 在截止前一定遇到一個能用的完整 frame。

5 Phase 2：日前 MILP 排程 變數、線性化與目標

VppMilpFormulator 把 72 小時排程建成混合整數線性規劃 (MILP)，以 PuLP + CBC 一次解出整個 horizon。

5.1 決策變數

變數	意義 (型別)
$P_{i,t} \geq 0$	設備 i 在 t 的總出力 (MWh, 連續)
$k_{j,i,t} \geq 0$	設備 i 在 t 供給需求 j 的能量； $j \in J_{chg}$ 時為對電池的充電量 (連續)
$u_{i,t}, \text{start}_{i,t}, \text{stop}_{i,t} \in \{0, 1\}$	機組開關機 / 啟動 / 停機 (線性化 $\min(1, P)$)
$\text{charge_b}_{i,t}, \text{discharge_b}_{i,t} \in \{0, 1\}$	電池充 / 放電旗標 (互斥, 線性化 C19)
$\text{SOC}_{i,t} \in [\text{stomin}_i, \text{stomax}_i]$	電池在 t 末的儲能狀態 (連續)
$\text{Sell}_t \geq 0$	t 時售電量 (連續)
$x_{j,t} \in \{0, 1\}$	需求 j 在 t 是否執行 (線性化 $\min(1, \sum_i k)$)

5.2 關鍵理論：規格的 $\min(1, \cdot)$ 如何被線性化

規格的式子大量使用 $\min(1, \sum_i k_{j,i,t})$ 與 $\min(1, P_{i,t})$ ，意思都是「有沒有在作用」的 0/1 指示。但 $\min$ 是非線性，MILP 不能直接吃。我們的做法是引入指示二元變數、再用線性不等式把它和原連續量綁定，這是把規格轉成可解模型的核心，也是「沒有新增超出語意的決策」的關鍵：

- **job 是否執行**：令 $x_{j,t} = \min(1, \sum_i k_{j,i,t})$ 。由 C1 (見下) $\sum_i k_{j,i,t} = w_j x_{j,t} : x = 1$ 時恰供 w_j 、 $x = 0$ 時供 0。於是凡是規格寫 $\min(1, \sum k)$ 的地方都換成 x 。

- **機組是否開機**：令 $u_{i,t} = \min(1, P_{i,t})$ 。由 C6 $outputmin_i u_{i,t} \leq P_{i,t} \leq outputmax_i u_{i,t}$ ： $P > 0 \Rightarrow u = 1$ 、 $u = 0 \Rightarrow P = 0$ 。成本 f_2 中的 $\min(1, P)$ 即以 u 取代。
- **開關機事件**：標準機組排程 (unit commitment) 做法 $start_{i,t} - stop_{i,t} = u_{i,t} - u_{i,t-1}$ 、 $start + stop \leq 1$ ，把「狀態變化」抓成事件變數，供最小開/關機 (C9/C10) 使用。
- **充放電互斥**：C19 規格是雙線性 $P_{i,t} \cdot \sum k = 0$ （非線性）；我們用 $charge_b + discharge_b \leq 1$ ，並由 C14 $P \leq dis \cdot discharge_b$ 、C15 $charge_in \leq chg \cdot charge_b$ 把功率綁到各自旗標，達到「同時段不能又充又放」。

設計原理：所有二元變數都只是既有連續量的「開關指示」，不新增任何超出規格語意的自由度；這讓模型在 CBC 下可解，同時保持與規格數學式等價。

5.3 目標函數與求解流程

目標即 §1.3 的 F 。實作中 $f_2 = \sum_{i \in I_g} \sum_t (coston_i u_{i,t} + costup_i P_{i,t})$ 、 $f_3 = -\sum_t \lambda_t Sell_t$ 。Level 1 的 MILP 只把週期 job 與充電 job 納入；零星/非週期由 Phase 3 以保留量處理，故靜態 MILP 求解時 $f_1 = 0$ ，非週期逾時罰則於 Phase 4 評估時才計入 objective_value。求得最佳解後，SchedulerResultExtractor 解析各變數、四捨五入、捨去近零值，輸出 schedule_result.json。

[對應評分 3-1]

6 Phase 2：全部 23 條限制式（規格式 + 實作 + 原理）

以下逐條給出規格的數學式、程式對應與設計原理，並標註對應評分項。 $\sum_i$ 預設對允許的設備集合求和。

6.1 任務與供需路由 (C1–C3, C5, C20)

[對應評分 2-1，5 分]

C1 啟動時收滿需求。

$$\sum_{i \in I} k_{j,i,t} = w_j \cdot \min\left(1, \sum_{i \in I} k_{j,i,t}\right), \quad \forall j \in J \setminus J_{chg}, \forall t. \quad (6)$$

實作 (C1_demand)： $\sum_i k_{j,i,t} = w_j x_{j,t}$ 。**設計原理**：用電需求是「整包」的，一個 job 在某時段若執行，就必須當格完整供滿 w_j MWh，不能只供一半。這把「執行」定義成離散的 0/1 事件 x ，是 C3/C5 等所有計數式的基礎。

C2 釋放前不可執行。

$$k_{j,i,t} = 0, \quad \forall j \in J \setminus J_{chg}, \forall i, \forall t < r_j. \quad (7)$$

實作： k 變數只在 $[r_j, \text{deadline}]$ 內建立，等價於釋放前恆為 0。**設計原理**：即時任務的釋放時間語意 job 還沒到達就不存在，不能提前供電。

C3 截止前累積執行恰 e 格。

$$\sum_{t=r_j}^{r_j+d_j-1} \min\left(1, \sum_i k_{j,i,t}\right) = e_j, \quad \forall j \in J_p \cup J_s. \quad (8)$$

實作 (C3_exec)： $\sum_{t \in [r_j, d_j]} x_{j,t} = e_j$ 。設計原理：硬截止的完成性：硬任務（週期、零星）必須在絕對截止前不多不少做滿 e 格。等式（而非 $\geq$ ）同時排除「沒做完」與「做超過」，後者會浪費電能。 [對應評分 3-2]

C5 非搶占任務須連續執行。

$$\sum_{t=r_j}^{H-1} \left| \min(1, \sum_i k_{j,i,t+1}) - \min(1, \sum_i k_{j,i,t}) \right| \leq 2, \quad \forall j \in J_{np}. \quad (9)$$

實作 (C5_rise/contiguous)：對視窗內每個 t 設「上升沿」二元 $\text{rise}_t \geq x_{j,t} - x_{j,t-1}$ ，並令 $\sum_t \text{rise}_t \leq 1$ 。設計原理：規格用「狀態變化次數 ≤ 2 」（恰一次 $0 \rightarrow 1$ 與一次 $1 \rightarrow 0$ ）表達「只有一個執行區塊」。我們等價地限制上升沿至多 1 次；配合 C3 已固定 $\sum x = e$ ，就唯一地強制成單一連續區塊。這正是非搶占（不可中斷）的數學定義。把 release 那格也算進可能的上升沿，否則「從 release 起一段 + 後面再來一段」會被漏判。

C20 設備供給不超過其出力。

$$\sum_{j \in J} k_{j,i,t} \leq P_{i,t} \quad (i \in I_g \cup I_r); \quad \sum_{j \in J \setminus J_{chg}} k_{j,i,t} \leq P_{i,t} \quad (i \in I_b), \quad \forall t. \quad (10)$$

實作 (C20_devalloc)：每設備每時段「分配出去的總和 $\leq$ 它的出力」。設計原理：能量守恆於設備層級。不能分配比實際發出更多的電。儲能那式排除 J_{chg} ，因為「對電池充電」是輸入而非該電池的輸出。

6.2 非週期任務的軟截止 (C4)

[對應評分 2-2，2 分]

$$\sum_{t=r_j}^{r_j+d_j-1} \min(1, \sum_i k) \geq e_j(1 - \text{Miss}_j), \quad \sum_{t=r_j}^{r_j+d_j-1} \min(1, \sum_i k) \leq e_j - 1 + e_j(1 - \text{Miss}_j), \quad (11)$$

$$\sum_{t=r_j}^H \min(1, \sum_i k) = e_j, \quad \forall j \in J_a. \quad (12)$$

設計原理：軟截止的可逾時語意：非週期任務最終一定要做完（第三式：在 H 前做滿 e 格），但允許晚於截止。前兩式用 Miss_j 把「截止前是否做滿 e 格」轉成 0/1：截止前做滿則 $\text{Miss} = 0$ ，否則 $\text{Miss} = 1$ 並進目標被罰 α 。本作業中非週期任務由 Phase 3 接收測試實際安排， Miss_j 由完成時刻是否晚於軟截止判定。

6.3 傳統機組（C6–C12）

[對應評分 2-3，7 分]

C6 出力上下限。 $outputmin_i \Delta t \min(1, P_{i,t}) \leq P_{i,t} \leq outputmax_i \Delta t \min(1, P_{i,t})$ 。實作 $outputmin_i u_{i,t} \leq P_{i,t} \leq outputmax_i u_{i,t}$ 。設計原理：火力機組一旦開機有最低出力（不能無限低、否則熄火），關機則出力為 0；用 u 同時表達「開機才有出力下限」與「關機強制歸零」。

C7 爬升率。 $P_{i,t} - P_{i,t-1} \leq RU_i \Delta t$ ， $P_{i,t-1} - P_{i,t} \leq RD_i \Delta t$ 。設計原理：熱機慣性：火力機組相鄰時段出力增/減幅有物理上限，不能瞬間跳變。 $t = 1$ 時以 $P_{i,0}$ 接續初始狀態。

C8 最低出力不可大於單格爬升能力。 $outputmin_i \leq RU_i \Delta t$ 。設計原理：可行性自洽：機組從關機（0）啟動到必須達到的最低出力 $outputmin$ ，一格內爬得上去才合理；否則一開機就違反爬升率，模型無解。這是對輸入資料的一致性條件。

C9 最小開機時間。 $\sum_{\tau=t}^{t+UT_i-1} \min(1, P_{i,\tau}) \geq UT_i \max(0, \min(1, P_{i,t}) - \min(1, P_{i,t-1}))$ 。實作以 $\sum_{s=t}^{t+UT_i-1} u_{i,s} \geq (UT_i) start_{i,t}$ 。設計原理：最小運轉時間：機組剛啟動就被要求連開 UT_i 格，避免頻繁開關造成設備損耗 這對應規格右側「啟動事件」乘以 UT_i 。

C10 最小關機時間。 對稱於 C9，實作 $\sum_{s=t}^{t+DT_i-1} (1 - u_{i,s}) \geq DT_i stop_{i,t}$ 。設計原理：最小停機時間：剛停機就得連關 DT_i 格，避免抖動。

C11 初始開機延續。 $\sum_{\tau=1}^{UT_i-TN_i} \min(1, P_{i,\tau}) = UT_i - TN_i$ 。實作：強制前 $\max(0, UT_i - TN_i)$ 格 $u = 1$ 。設計原理：跨排程邊界的狀態承接 若排程起點前機組已連開 TN_i 格但未滿 UT_i ，剩餘的最小開機時間要補足，不能一開始就停。

C12 初始關機延續。 對稱：強制前 $\max(0, DT_i - TF_i)$ 格 $u = 0$ 。設計原理：同理承接「已關機未滿 DT_i 」的狀態。

6.4 再生能源（C13）

[對應評分 2-4，1 分]

$$0 \leq P_{i,t} \leq renewmax_i \cdot \Delta t \cdot renewforecast_{i,t}, \quad \forall i \in I_r, \forall t. \quad (13)$$

設計原理：太陽能出力受天候上限約束，不可調度向上：只能用到「額定 × 當時段日照預測比例」這麼多。它無成本（不進 f_2 ），所以最佳化會優先用滿再生能源、再考慮火力。Level 2 會把這條折減（R1）。

6.5 儲能（C14–C19, C21）

[對應評分 2-5，7 分]

C14 放電上限。 $0 \leq P_{i,t} \leq dis_i \Delta t$ 。實作含旗標 $P \leq dis_i discharge_b$ 。設計原理：電池放電功率（C-rate）上限；綁旗標同時服務 C19 的互斥。

C15 充電上限。 $0 \leq \sum_{j \in J_{chg}: b(j)=i} \sum_{i' \in I_g \cup I_r} k_{j,i',t} \leq chg_i \Delta t$ 。實作含旗標 $\leq chg_i charge_b$ 。設計原理：充電功率上限；充電量只統計來自機組/再生能源（呼應 C21）。

C16 SOC 平衡。

$$SOC_{i,t} = SOC_{i,t-1} + \sum_{j \in J_{chg}: b(j)=i} \sum_{i' \in I_g \cup I_r} k_{j,i',t} - P_{i,t}, \quad \forall i \in I_b, \forall t. \quad (14)$$

設計原理：儲能的狀態方程（能量守恆）：本時段末的存量 = 前一格存量 + 充入 - 放出。它讓電池成為跨時段搬移能量的「緩衝器」（低價/高再生時充、高價/缺電時放）。Level 2 在此處加上效率與自放電（R2-R4）。 $t = 1$ 以 $SOC_{i,0}$ 接續。

C17 SOC 上下限。 $stomin_i \leq SOC_{i,t} \leq stomax_i$ 。實作：直接設為 SOC 變數的上下界。**設計原理：**電池不能過充過放（保護電芯、保留備用），這是儲能可用容量區間。

C18 放電不超過可用存量。 $P_{i,t} \leq SOC_{i,t-1} - stomin_i$ 。**設計原理：**當下可放的能量上限 只能放出「高於最低存量」的部分。它比 C17 更即時：C17 是事後狀態界，C18 是事前的放電量界，兩者一起避免穿底。Level 2 改為扣效率/自放電（R5）。

C19 不可同時充放。 $P_{i,t} \cdot \sum_{j \in J_{chg}: b(j)=i} \sum_{i'} k_{j,i',t} = 0$ 。實作：charge_b + discharge_b ≤ 1 （見 §5.2）。**設計原理：**物理互斥：同一時段電池不可能又充又放（否則等於憑空製造往返損耗或套利）。規格的雙線性式以兩個旗標線性化。

C21 充電只能由發電設備供應。 $k_{j,i,t} = 0, \forall j \in J_{chg}, i \notin I_g \cup I_r$ 。實作：充電 job 的 k 變數只對機組/再生能源建立。**設計原理：**禁止電池互充 充電能量必須來自真正的發電源，不能由另一顆電池放電來灌，否則會出現無意義的能量在電池間搬運。

6.6 售電與供需平衡（C22, C23）

[對應評分 2-6，1 分；2-7，4 分]

C22 售電非負。 $Sell_t \geq 0$ 。實作：Sell 變數下界 0。**設計原理：**本模型只賣電不買電；售電量不可為負。

C23 每小時供需平衡（模型骨幹）。

$$\sum_{i \in I} P_{i,t} = \sum_{j \in J \setminus J_{chg}} \sum_i k_{j,i,t} + \sum_{j \in J_{chg}} \sum_{i \in I_g \cup I_r} k_{j,i,t} + Sell_t, \quad \forall t. \quad (15)$$

設計原理：整個系統的能量守恆（瓦時平衡）：每一格的總發電，必須恰好等於「用電需求 + 充電消耗 + 賣到市場」。它把所有設備與所有需求綁在一起，是模型的脊椎；接收測試「轉移剩餘、扣售電」之所以不破壞模型，正是因為它讓 C23 兩邊等額變動。評分 2-7 按「違反幾小時」扣分，故我們逐格驗證 0 違反。

7 Phase 3：接收測試（評分 4-1 ~ 4-3）

AcceptanceTester 在 MILP 解出後執行（整合進 RTScheduler.run()）。

7.1 保留量的數學定義（4-1 方法）

[對應評分 4-1]

MILP 解完後，每格沒賣掉而原本要送市場的剩餘，就是可挪用的**保留量**。它可拆解為各設備的 spare：

$$spare_{i,t} = P_{i,t} - \sum_j k_{j,i,t}, \quad \sum_i spare_{i,t} = Sell_t \quad (\text{由 C23}). \quad (16)$$

接受一個 job 時，把它每格需求 w 從 spare 轉入 $k_{\text{job},i,t}$ ，並同額減少 Sell_t 。設計原理：因為「轉進去的量」與「少賣的量」相等，C23 兩邊同時減同一個數，平衡仍成立；C20（設備上限）也因為只動用了 spare ($P - \sum k \geq 0$ 的部分) 而不被破壞。所以接收測試不需要重解 MILP，只是在既有最佳解的「縫隙」裡塞工作。這對應評分 4-1 要求的「可用空檔/資源餘裕檢查 + 接受後如何更新狀態」。

7.2 接受/拒絕判斷規則（4-2 合理性）

[對應評分 4-2]

給定 job 的視窗 $[r, r + d - 1]$ （零星）或 $[r, H]$ （非週期）、執行格數 e 、每格需求 w ：

- 搶占 ($\text{preempt} = 1$)：在視窗內挑出所有「剩餘保留 $\geq w$ 」的時槽，若數量 $\geq e$ 則取最早 e 個；否則拒絕。
- 非搶占 ($\text{preempt} = 0$)：找一個長度 e 的連續視窗，其中每一格剩餘保留 $\geq w$ ；找到即接受，否則拒絕。
- 零星（硬截止）：找不到合格時槽 $\Rightarrow$ 拒絕 (reject)。
- 非週期（軟截止）：盡量排到 H 前完成；完成時刻晚於軟截止則記 miss。

設計原理：硬截止任務「寧可拒絕、不可逾時」 接受後保證準時是即時系統 admission control 的精神；非搶占要連續視窗，是直接對應 C5 的線上版本。每筆決策（接受/拒絕、時槽、完成時刻、理由）都寫入 `acceptance_test_log.json`，逐 job 給出可被質詢的理由（例如「 $[28, 33]$ 內找不到連續 2 格皆 $\geq w$ 的保留」）。

7.3 Sporadic value rate (4-3)

[對應評分 4-3]

$$\text{value rate} = \frac{\sum_{j \in J_s, \text{截止前完成}} e_j}{\sum_{j \in J_s} e_j} \in [0, 1]. \quad (17)$$

評分採分段給分：0 得 0 分、 $0 < \text{rate} < 0.4$ 得 1、 $0.4 \leq \text{rate} < 0.7$ 得 2、 ≥ 0.7 得 3。

設計原理：規格刻意不以「接受幾個」而以「在硬截止前實際完成的執行格總長」衡量零星任務的價值 因為接受了卻沒做完毫無意義。本次 6 個零星全部接受且準時完成， $\text{value rate} = 1.0$ （滿分段）。

7.4 本次靜態接收測試結果

零星 $s1-s6$ 全部接受 ($\text{rate} = 1.0$)；非週期 $a1-a10$ 全部排入，但 $a2$ （完成於 21 $>$ 軟截止 17）與 $a5$ （完成於 46 $>$ 軟截止 42）逾時，故靜態軟逾時率 $= 2/10 = 0.2$ 。設計原理：靜態日前計畫雖一次看到全部任務，但只承諾一份排程、保留量有限，無法為較晚到的軟任務都留足空檔 這正是 Level 2 動態法要改善的地方。

8 Phase 4：效能評估指標（評分 5-1 ~ 5-5）

Evaluator 讀入排程與輸入檔，輸出 `evaluation_results.json`。令 C_j 為 job j 的完成時刻（最後一個有 $k_{j,t} > 0$ 的 t ）、 d_j 絕對截止、 r_j 釋放。

- **5-1 硬截止逾時率** [對應評分 5-1]： $\frac{\#\{\text{週期} + \text{零星 job 逾時或被拒}\}}{\#\text{硬截止 job}}$ 。設計原理：硬任務只要 $C_j > d_j$ 或被拒就算 miss；它是即時系統最重要的正確性指標，本次 = 0.0。
- **5-2 軟截止逾時率** [對應評分 5-2]： $\frac{\#\{\text{非週期 miss}\}}{\#\text{非週期 job}}$ 。設計原理：軟任務逾時不算違規但要記錄；靜態 0.2、動態 0.0。
- **5-3 Tardiness** [對應評分 5-3]： $T_j = \max(0, C_j - d_j)$ ，回報 average / max。設計原理：逾時的「嚴重程度」 不只看漏沒漏，還看晚多久。靜態 avg 0.15、max 4；動態皆 0。
- **5-4 Response time** [對應評分 5-4]： $R_j = C_j - r_j$ ，回報 average / max。設計原理：即時系統的回應性 從釋放到完成多久。越短代表越敏捷；靜態 avg 3.63、動態 2.65。
- **5-5 Completion-time jitter** [對應評分 5-5]：對同一週期任務的各 instance，取完成時刻的峰對峰值 $\max(C) - \min(C)$ ，再對所有任務平均。設計原理：週期任務完成時刻的穩定度 jitter 小代表每期都在差不多的時刻完成，對需要規律性的製程友善。

此外輸出 $\text{generator_cost} = f_2$ 、 $\text{market_revenue} = \sum_t \lambda_t \text{Sell}_t$ 、 $\text{objective_value} = \alpha \cdot (\text{軟逾時數}) + f_2 - \text{售電收益}$ ，作為三目標的量化結果。

9 日前保留策略與目標權衡分析（評分 6-1, 6-2）

9.1 保留策略：留多少、留在哪、怎麼用（6-1）

[對應評分 6-1]

保留策略以一條售電下限表達（重用 Sell，不新增變數）：

$$\text{Sell}_t \geq R_t, \quad \forall t, \quad (18)$$

其中 R_t 是「該時段要為未來零星/非週期任務留下的可挪用剩餘」。動態法中 R_t 由尚未完成的已接受 job 的需求總和動態算出（見 §11）：

$$R_t = \min \left(\text{cap}, \sum_{j: t \in \text{job } j \text{ 的保留時槽}} w_j \right). \quad (19)$$

設計原理：留多少：恰好等於已接受未完成任務在該格的能量需求和（再加一個上限避免過度保留拖垮可行性）。留在哪：留在這些任務各自的執行視窗（每個 job 取最早的剩

餘格)。怎麼用：任務到達時若 R_t 撐得住（再最佳化仍可行），就接受並把保留量在區塊凍結時路由進 k 。本質上這是把「接收測試」變成「最佳化的可行性問題」能保留得住才接。

9.2 三目標的權衡（6-2）

[對應評分 6-2]

三個目標彼此衝突，無法同時最佳（Pareto 取捨）：

- **提高接收率（降 miss， $f_1 \downarrow$ ）** $\Rightarrow$ 需把更多剩餘留著（ $R_t \uparrow$ 、Sell 被綁住） $\Rightarrow$ 要嘛多開機組（ $f_2 \uparrow$ ）、要嘛少在尖峰賣（收益 $\downarrow$ ）。
- **衝高售電收益** $\Rightarrow$ 把剩餘都在高價時段賣掉 $\Rightarrow$ 留給臨時任務的保留變少 $\Rightarrow$ miss 風險上升。
- **壓低機組成本** $\Rightarrow$ 少開貴機組 $\Rightarrow$ 可挪用剩餘變少 $\Rightarrow$ 同樣與前兩者衝突。

設計原理： $\alpha = 10000$ 的設計讓這個權衡有明確的「匯率」：只要少漏一個非週期任務值得多花到一萬元的成本，最佳化就會選擇保留。實測 (§12) 顯示動態法用 +15025 的機組成本，換到「軟逾時由 2 降到 0（省 $2\alpha = 20000$ 罰則）」且售電收益反而 +8854，總目標明顯更佳 數字上印證了「提升接收率須以成本/收益換取」的權衡論述（這正是評分 6-2 要求的「以實際數據說明三目標權衡，並納入 sporadic value rate」）。

10 Level 2-A：放寬假設建模 R1–R10（評分 3-1）

我們放寬三個 Level 1 假設 **假設 11**（再生能源預測必準）、**假設 12**（理想儲能）、**假設 5**（無工作優先序）並嚴格遵守規格：**可修改符號、可加參數，但不得新增決策變數**。下列每條都只用 Level 1 既有變數 $P, k, SOC, \text{Sell}, x$ 表達。所有參數集中於 runtime_config.json；全取預設（ $\eta = 1, \sigma = 0, \beta = 0, N = \infty, \varphi = 1, c_{age} = 0$ ）時，模型與 Level 1 **完全相同**（這是「一個放寬限制式得 1 分」能逐條驗證的前提）。

10.1 放寬參數（本次採用值）

參數	符號	值	假設	意義
charge_efficiency	η_c	0.95	12	充電能量僅 η_c 進入電芯
discharge_efficiency	η_d	0.95	12	放出 P 需從電芯抽 P/η_d
self_discharge_rate	σ	0.002	12	每格自放電比例
cycle_limit	N	3.0	12	全期最大等效循環數
soc_power_floor	φ	0.5	12	SOC 極端時最低功率比例
aging_cost	c_{age}	2.0	12	每放電 1 MWh 老化成本 (\$)
renewable_uncertainty_margin	β	0.15	11	預測折減保留邊際
precedence	—	自動	5	有序工作對 (a, b)

10.2 逐條：數學式 + 物理/工程原理

R1（C13'）再生能源不確定性折減—假設 11。

$$P_{i,t} \leq \text{renewmax}_i \cdot \text{renewforecast}_{i,t} \cdot (1 - \beta) \cdot \Delta t. \quad (20)$$

設計原理：物理面：日前 24–72 小時的日照預測含系統性與隨機誤差（雲量演變、氣溶膠、數值預報解析度），太陽能日前預測的正規化平均絕對誤差（nMAE）典型落在 10–20%；而再生能源**無燃料成本、不進 f_2** ，最佳化必然把它用到上限（C13 取等號）。一旦日前押滿點預測、實際出力短缺，C23 供需平衡即被破壞，只能臨時加開高成本機組、甚至缺電而漏掉硬截止。**原理面：**本式把「點預測」改成「保守下界情境 $\text{forecast} \cdot (1 - \beta)$ 」，是最簡形式的穩健最佳化（robust optimization）/ 箱型不確定集 以一個確定性折減代替顯式情境集合，不引入新變數即保留 MILP 結構。 β 是**雙向取捨**的旋鈕：太小（趨近 0）等於完全相信預測、留不住餘裕；太大則浪費免費綠電、逼出更多火力使 f_2 上升。取 $\beta = 15\%$ 對齊典型日前 PV nMAE。**與動態法的關係：**動態層僅對「尚未實現的尾段」套用折減，已執行的承諾區塊改用實測 PV (§11)，故 β 是日前的保守度、而非事後懲罰。 $\beta = 0$ 還原 C13。

R2/R3/R4（C16'）真實 SOC 平衡：往返效率 + 自放電—假設 12。

$$SOC_{i,t} = (1 - \sigma) SOC_{i,t-1} + \eta_c \text{charge_in}_{i,t} - \frac{1}{\eta_d} P_{i,t}. \quad (21)$$

設計原理：理想電池的能量守恆（C16）假設「充多少存多少、放多少送多少、靜置不漏」。真實電池有三個獨立的物理損耗，各記 1 分：

- **R2 充電效率 η_c ：**充電要經整流 / 變流與內阻發熱，灌入的 charge_in 只有 $\eta_c = 95\%$ 真正進入電芯，餘者以熱散失 故存量增加項是 $\eta_c \text{charge_in}$ 而非 charge_in。

- R3 放電效率 η_d ：要在匯流排送出 P ，須從電芯多抽 P/η_d （同有變流 / 內阻損耗）故存量扣除項是 $\frac{1}{\eta_d}P$ 而非 P 。
- R4 自放電 σ ：靜置時經並聯漏電與 BMS 寄生負載緩慢漏電，前一格存量先打折 $(1 - \sigma)$ 才結轉，呈幾何衰減。

三者合起來把理想電池變成往返效率 $\eta_c\eta_d \approx 0.90$ 的真實鋰電池。經濟意涵：低買高賣的套利唯有在「價差 > 往返損耗」時才划算。理想電池（ $\eta = 1, \sigma = 0$ ）會無代價地頻繁充放、扭曲排程，加入損耗後最佳化才會節制無謂的循環。三項係數皆為常數，模型仍為線性（MILP）。 $\eta_c = \eta_d = 1, \sigma = 0$ 還原 C16。

R5（C18'）放電受真實可用存量限制。

$$\frac{1}{\eta_d}P_{i,t} \leq (1 - \sigma)SOC_{i,t-1} - stomin_i. \quad (22)$$

設計原理：原理面：放電當下從電芯抽取的能量是 P/η_d （含放電損耗），其上限是「自放電後、再扣掉不可動用底線 $stomin$ 的可用存量」 $(1 - \sigma)SOC_{t-1} - stomin$ 。**為何不可省略：**C16 是事後的狀態結轉、C18 是事前的單格放電量界；若 C18 仍用理想式 $P \leq SOC_{t-1} - stomin$ 而 C16 已扣效率，兩式對「究竟抽走多少電芯能量」的記帳就不一致，SOC 可能在某格穿底或使模型數值不自洽。R5 讓兩處同步採用 $1/\eta_d$ 與 $(1 - \sigma)$ ，保證放電量界與狀態方程一致、電池不穿底。效率取 $1, \sigma = 0$ 時還原 C18。

R6 循環/吞吐上限—假設 12。

$$\sum_t P_{i,t} \leq N(stomax_i - stomin_i). \quad (23)$$

設計原理：物理面：鋰電池的容量衰退主要由累積吞吐量（等效完整循環數）驅動，而非僅看日曆年限；每一次深度充放都磨耗極板與電解液。**原理面：**把全期總放電量 $\sum_t P_{i,t}$ 限制在 N 個「滿可用容量 $(stomax - stomin)$ 」之內，等價於全期最多 $N = 3$ 次等效完整循環（72 小時約每日 1 次，貼近保固級使用率）。這是一條硬上限，防止最佳化為了壓成本 / 衝售電而把電池操到提早報廢。它與 R9 的軟成本一硬一軟、互補：R6 給不可逾越的天花板，R9 給連續的邊際訊號。 $N = \infty$ 還原無限制。

R7 SOC 相依放電功率—假設 12。

$$P_{i,t} \leq dis_i \Delta t \left[\varphi + (1 - \varphi) \frac{SOC_{i,t-1} - stomin_i}{stomax_i - stomin_i} \right]. \quad (24)$$

設計原理：物理面：真實電池接近空電時可放功率下降。端電壓隨 SOC 下垂、內阻上升、BMS 觸發低電量限流保護，使瞬時可放功率隨 SOC 降低而衰減。**原理面：**方括號是一個隨 SOC 線性的折減因子，從滿電的額定 dis_i （括號 = 1）線性遞減到空電的 $\varphi \cdot dis_i$ （括號 = φ ），以一次內插近似實際的功率-SOC 曲線、又維持線性可解。本次 $\varphi = 0.5$ 表示空電時功率減半。 $\varphi = 1$ 還原定額放電界（C14）。

R8 SOC 相依充電功率—假設 12。

$$charge_in_{i,t} \leq chg_i \Delta t \left[\varphi + (1 - \varphi) \frac{stomax_i - SOC_{i,t-1}}{stomax_i - stomin_i} \right]. \quad (25)$$

設計原理：物理面：對稱地，接近滿電時可充功率下降 鋰電池充到高 SOC 後進入定電壓 (CV) 階段，改以逐步收斂的涓流充電以免過充。**原理面：**折減因子改用「離滿電的距離」 $stomax - SOC_{t-1}$ ，於滿電時降到 $\varphi \cdot chg_i$ 、空電時為額定 chg_i ，以線性內插近似鋰電池的 CC-CV（先定流後定壓）充電輪廓。 $\varphi = 1$ 還原定額充電界 (C15)。

R9 老化成本進目標—假設 12。

$$F += \sum_{i \in I_b} \sum_t c_{age} P_{i,t}. \quad (26)$$

設計原理：原理面：把電池損耗內部化成目標函數中的邊際成本 每放電 1 MWh 計折舊 $c_{age} = 2 \$$ 。如此最佳化會在「循環電池」與「多燒一點燃料」之間做經濟權衡：唯有當套利收益或避免逾時的罰則 > 老化成本時，才動用電池放電，否則寧可讓機組多供一些。與 R6 的分工：R6 是吞吐量的硬天花板（不可逾越、二值式的禁止），R9 是連續的軟訊號（每一單位吞吐都計價、可被收益超過而動用），兩者共同塑形電池的使用強度與時機。 $c_{age} = 0$ 時不影響目標、還原 Level 1。

R10 工作優先序—假設 5。

$$e_a x_{b,t} \leq \sum_{s < t} x_{a,s}, \quad \forall t \in b \text{ 的視窗}. \quad (27)$$

設計原理：原理面：有序對 (a, b) 表示「 a 全部 e_a 格做完前， b 不得啟動」。當 a 在 t 之前累積執行格數 $\sum_{s < t} x_{a,s} < e_a$ 時，右側 $< e_a$ ，左側被迫 $x_{b,t} = 0$ ；一旦 a 做滿 e_a 格、右側達 e_a ， b 才解禁。這是經典排程的前置依賴 / DAG 拓樸序 (precedence constraint)，對應真實流程的工序相依，例如「先抽水才能曝氣」、「先預熱才能反應」。**符合規格：**僅重用既有的活動二元 x 、無新增決策變數；未明確設定時，程式自動挑一組可行的非平凡對（本次為 $p2_0 \rightarrow p4_0$ ），確保此限制真的被觸發而非擺設。空集合 $\Rightarrow$ 無此限制、還原 Level 1。

合計 R1–R10 共 10 條放寬限制，逐條對應評分 3-1 的「一個限制式 1 分、上限 10 分」，且皆未新增決策變數。另有「保留下限 $Sell_t \geq R_t$ 」（§9）支援接收測試與動態法。

11 Level 2-B：進階動態排程（評分 8-1 ~ 8-3）

11.1 方法選擇與理由（8-1）

[對應評分 8-1]

方法：滾動視窗再最佳化 (Receding-horizon / Model Predictive Control, MPC)。

Level 1 在 $t = 0$ 一次解完整個 $[1, 72]$ ；AdvancedScheduler 則沿時間前進，在觸發點對「剩餘 horizon」重新最佳化，因應靜態計畫看不到的動態資訊（再生能源實際值、臨時任務到達）。**設計原理：**選 MPC 而非完全重排或啟發式，是因為它同時兼顧最佳性與可解性：每次都解一個真正的 MILP（最佳、滿足全部限制），但只解「還沒執行的尾段」，且把已執行段釘住，所以規模隨時間縮小、線上可解。這對應評分 8-1 要求的「清楚定義更新時機、判斷規則、整體流程，並說明選擇理由」。

核心機制：

- **狀態承接** `pin_prefix`：每次觸發時把已執行的 $[1, t_0)$ 釘成已承諾值(連續變數 P, k, SOC, Sell 加上由其推導的前綴二元 $u, \text{charge_b}, \text{discharge_b}, x$)，只重解 $[t_0, 72]$ 。**設計原理**：釘住連續狀態就能讓機組開關機時長、爬升位置、SOC **正確跨界承接** (Level 1 的限制式本就由這些連續量推導二元)；再固定前綴二元，solver 的 `presolve` 便能把凍結段整段消去，使每次再解都便宜。
- **觸發時機 (更新規則)**：(a) 週期節奏 `reopt_interval` = 24 格；(b) 每個零星任務釋放 (硬任務需即時審核)；(c) 至多 3 個「實際 PV 與預測偏差最大」的時點。非週期任務於下一個節奏邊界揭露，以控制再解次數。
- **再生能源實現**：以種子化隨機模型 (`noise std = 0.2, seed = 42`) 在預測周圍生成「實際」PV。已承諾區塊用實際值、未見尾段用折減預測 (R_t) **對已承諾精確、對未來保守**。
- **線上接收 = 保留可行性**：任務於釋放時揭露，能否接受取決於再最佳化「能否在其執行視窗內保留足夠可挪用剩餘 ($\text{Sell} \geq R_t$)」。若無解就丟掉最低優先的任務 (軟先於硬、重的先丟) 再重解；硬任務不可行即拒絕、軟任務記 `miss`。被接受者隨區塊凍結被路由進排程，故接受的硬任務必在截止前完成。

每次再解使用 `MIP gap` (`gap_rel` = 0.20) 與時間上限 (`time_limit` = 20s)、單執行緒，確保線上再最佳化在秒級可解且結果可重現。

11.2 排程結果正確性 (8-2)

[對應評分 8-2]

本次共 9 輪再最佳化、9 次 MILP 解 (單執行緒、固定種子、可重現)。任務隨時間揭露並被接受 (見 §12 表)。最終所有 38 個週期 job、6 個零星 job 全部於硬截止前完成、10 個非週期 job 全部於軟截止前完成。逐格驗證：C23 平衡 0 違反、SOC 維持於 $[\text{stomin}, \text{stomax}]$ 0 違反、放寬儲能/機組/再生能源限制每輪皆成立，過程中無 job rejection、無 miss、無重排失敗。**設計原理**：評分 8-2 要求「檢查最終結果是否滿足主要限制」並交代每輪 jobs 安排與接受/拒絕情形 我們以 `python3 -m src.validator --level 2` 用放寬後的限制式逐項重驗 (驗證「實作」而非僅「描述」)。

12 實際執行結果與靜態 vs 動態比較 (評分 8-3)

12.1 動態排程的 9 輪再最佳化

觸發 t_0	承諾區間	本輪揭露	結果
1	[1, 1]	(無)	建立初始計畫
2	[2, 13]	s1, a1, a2	全部接受並路由
14	[14, 24]	s2, a3	全部接受
25	[25, 27]	a4	接受
28	[28, 39]	s3, a5	全部接受
40	[40, 48]	s4, a6	全部接受
49	[49, 52]	a7	接受
53	[53, 65]	s5, a8, a9	全部接受
66	[66, 72]	s6, a10	全部接受

9 輪皆無拒絕、無再排程失敗。

12.2 靜態 (L1) vs 動態 (L2) 效能比較 (8-3)

[對應評分 8-3]

指標	靜態 L1	動態 L2	差異 Δ
objective_value	-69 410.4	-83 238.8	-13 828.4 (更佳)
generator_cost	296 403.6	311 428.9	+15 025.3
market_revenue	385 814.0	394 667.8	+8 853.8
hard_deadline_miss_rate	0.0	0.0	—
soft_deadline_miss_rate	0.2	0.0	-0.2
average_tardiness	0.15	0.0	-0.15
max_tardiness	4	0	-4
average_response_time	3.63	2.65	-0.98
sporadic_value_rate	1.0	1.0	—

解讀 (呼應 §9 的權衡)：

- 靜態日前計畫雖一次看到所有任務，但只承諾一份排程，漏掉 2 個非週期任務(a2、a5，軟逾時率 0.2)。動態法隨任務到達再最佳化並為其保留容量，10 個全部準時(軟逾時率 0.0)，平均回應時間由 3.63 降到 2.65。
- 消除 2 個軟逾時 = 省下 $2\alpha = 20\,000$ 罰則，主導目標改善 ($-69\,410 \rightarrow -83\,239$)。

- 代價清晰：保留容量並補償實際再生能源短缺使機組成本上升（+15 025）；較大的承諾發電同時讓售電收益增加（+8 854）。扣掉逾時罰則後，動態法在總目標上明顯更佳，與理論權衡一致。

13 程式碼結構與執行方式

```
src/
  main.py           管線進入點 (run_level2 = L1 + L2 + 比較)
  config.py         路徑與常數
  validator.py       自我檢核 (純標準庫)
  advanced_scheduler.py Level 2 滾動視窗動態排程
  generator/        Phase 1 任務生成
  rt_scheduler/      Phase 2+3: formulator / acceptance_tester /
                    relaxation / expander / extractor
  evaluator/        Phase 4 評估
  model/            Pydantic 資料模型
input/  processor_settings.json, price_72hr.json
output/ task_set.json, schedule_result*.json, evaluation_results*.json,
        acceptance_test_log*.json, dynamic_run_log.json
runtime_config.json Level 2 放寬 + 動態節奏參數
```

```
# 環境
pip install pulp pydantic          # 或 poetry install
# Level 1 完整管線 (生成 -> 排程 -> 評估)
python -m src.main
# Level 2 比較管線 (生成 -> 靜態 -> 動態 -> 印出比較)
python -c "from src.main import run_level2; run_level2()"
# 只跑動態排程 / 自我檢核
python -m src.advanced_scheduler
python3 -m src.validator             # Level 1
python3 -m src.validator --level 2  # Level 2
```

14 備詢速查表：評分項 → 數學式 → 一句原理

報告前最後複習用。被問到某項時，照「數學式 + 原理」回答即可。

14.1 Level 1 (80 分)

項	數學式 / 一句原理
1-1~1-3	欄位 $(r, p, e, d, w, preempt)$; $6 \leq J_p \leq 10$; 展開 > 30 。確保問題規模與工作量。
1-4	$1 \leq r \leq p, 6 \leq p \leq 24 (\geq 3 \text{ 種}), 1 \leq e \leq 4, e \leq d \leq p, 6 \leq w \leq 18$ 。參數合理且多樣。
1-5	$D_W = \sum e/p \geq 0.7$ 。利用率（負載）下限，保證中高負載。
1-6	$\geq 20\%$ 有 $d = e$ 。零鬆弛任務，逼出最緊迫的硬截止。
1-7	≥ 2 個 $e \neq 1$ 非搶占。讓 C5 連續區塊（阻塞）真的被觸發。
1-8	$f \geq \max e, H \bmod f = 0, 2f - \gcd(f, p) \leq d$ 。frame-based 時鐘驅動排程三條件。
2-1	C1 $\sum k = wx$; C2 釋放前 $=0$; C3 $\sum x = e$; C5 上升沿 ≤ 1 ; C20 $\sum k \leq P$ 。任務完成性與守恆。
2-2	C4 用 Miss 抓「截止前是否做滿」。軟截止可逾時但要記錄。
2-3	C6 出力界、C7 爬升、C8 自洽、C9/C10 最小開關機、C11/C12 初始承接。機組物理。
2-4	C13 $P \leq cap \cdot forecast \cdot \Delta t$ 。再生能源受天候上限、不可向上調度。
2-5	C14/15 充放界、C16 SOC 平衡、C17/18 存量界、C19 互斥、C21 不可互充。儲能狀態方程。
2-6	C22 $Sell \geq 0$ 。只賣不買。
2-7	C23 $\sum P = \sum k + Sell$ 。每小時能量守恆，模型骨幹。
3-1~3-3	JSON 輸出； $\sum x = e$ 且每格供滿 w ; $C_j \leq d_j$ 否則本項 0 分。週期 job 全準時。
4-1	$spare = P - \sum k, \sum spare = Sell$ 。在最佳解縫隙塞工作、扣售電保平衡，不重解。
4-2	硬任務找不到時槽即拒、非搶占需連續視窗。admission control：接受即保證準時。
4-3	$rate = \sum_{\text{準時}} e / \sum e = 1.0$ 。以「完成的執行格」而非「接受數」計價值。
5-1~5-5	硬/軟 miss rate ; $T_j = \max(0, C_j - d_j)$; $R_j = C_j - r_j$; jitter = $\max C - \min C$ 。
6-1	$Sell_t \geq R_t, R_t = \sum w_j$ （已接受未完成）。留多少/哪/怎麼用。
6-2	降 miss $\Rightarrow$ 多保留 $\Rightarrow f_2 \uparrow$ 或收益 $\downarrow$ 。加權和 (α) 下的 Pareto 取舍。

14.2 Level 2 (80~100 分)

項	數學式 / 一句原理
3-1 (R1)	$P \leq cap \cdot fc \cdot (1 - \beta)$ 。預測會高估 $\Rightarrow$ 保守折減 (robust)。
3-1 (R2-R4)	$SOC_t = (1 - \sigma)SOC_{t-1} + \eta_c \text{in} - P/\eta_d$ 。充/放效率 + 自放電。
3-1 (R5)	$P/\eta_d \leq (1 - \sigma)SOC_{t-1} - stomin$ 。放電受真實可用存量限。
3-1 (R6)	$\sum_t P \leq N(stomax - stomin)$ 。等效循環次數上限 (壽命)。
3-1 (R7/R8)	$P \leq dis [\varphi + (1 - \varphi) \frac{SOC - stomin}{usable}]$ (充電對稱)。SOC 相依功率 (CC-CV)。
3-1 (R9)	$F += \sum c_{age} P$ 。老化成本進目標，循環 vs 燒燃料權衡。
3-1 (R10)	$e_a x_{b,t} \leq \sum_{s < t} x_{a,s}$ 。前置依賴 (precedence)。重用 x ，無新變數。
8-1	滾動視窗 MPC：pin_prefix 凍結前綴、只重解尾段；觸發 = 節奏 + 零星到達 + PV 偏差。
8-2	9 輪 9 解；硬截止全準時、C23/SOC 0 違反。正確性逐格驗證。
8-3	軟 miss $0.2 \rightarrow 0$ (省 2 萬罰則)，成本 +1.5 萬、收益 +0.9 萬，總目標更佳。

第二部分 報告講稿（明日對老師報告）

本講稿提供「可逐字唸的完整版」。每張投影片分三塊：【講稿】是完整的口語稿，可直接照唸；【重點提示】是上台時的動作與不可漏的關鍵句；【被追問時】是常見追問的即答。完整唸約 15–18 分鐘；若時間緊，可只唸各【講稿】的首尾兩三句、把中段細節留到被問再補。被追問更深細節時，翻到第一部分對應小節或 §14 備詢速查表。

投影片 1：開場與題目定位（約 1.5 分鐘）

講稿：老師好，我們這組的題目是「虛擬電廠的即時排程系統」，英文是 Virtual Power Plant Real-Time Scheduling，簡稱 VPP-RTS。我先用一分多鐘說明，我們是怎麼把一個電力調度的問題，轉換成這門課的即時排程問題。所謂虛擬電廠，就是把分散在各處的發電與儲能資產——傳統的火力機組、太陽能、還有電池——整合成一個可以被統一調度的電廠。我們做的關鍵對應有五個：第一，把每一個發電或儲能設備，看成即時系統裡的「處理器」，這次我們有兩部火力機組、兩座太陽能、兩顆電池；第二，把每一個有時間限制的用電需求，看成「任務」，分成週期、零星、非週期三類；第三，一個任務的「執行」，就是在某個時段供應它所需要的電能；第四，處理器的容量，對應到設備的出力上限、電池的 SOC、還有機組的爬升率；第五，最後產出的「排程」，就是決定哪一個設備、在哪一個時段、供應多少電、又賣多少電到電力市場。時間軸我們切成 72 格、每格一小時，採用的是時鐘驅動的靜態排程，也就是在 $t = 0$ 的時候，一次把整個 72 小時的供電表算定。在這個框架下，我們要同時滿足三個彼此衝突的目標：第一，盡量不要漏掉任何一個非週期任務；第二，降低火力機組的發電成本；第三，提高賣電到市場的收益。這三個目標的單位不一樣——漏任務是「個數」、成本跟收益是「金額」——所以我們用一個罰係數 $\alpha = 10000$ ，把「漏一個任務」折算成金額，才能把三項放進同一條最小化的式子。這個 α 的設計，待會在 Level 2 的權衡分析會是關鍵的伏筆。

- 先指著 §1.1 的對應表，由「即時系統概念」逐列唸到「VPP 對應」，幫老師建立直覺。
- 強調「72 格、時鐘驅動、一次算定」就是靜態排程，這是和 Level 2 動態法對照的基準。
- 點出三目標彼此衝突， $\alpha = 10000$ 是把「個數」折算成「金額」的匯率，先埋伏筆。

被追問時：三類任務差在哪？週期任務按固定週期重複、硬截止；零星任務隨機到達、硬截止；非週期任務隨機到達、軟截止（可逾時但要記錄）。

投影片 2：四階段管線（約 1 分鐘）

講稿：接著看系統的四階段管線，整個流程由 `src/main.py` 串起來。Phase 1 是任務生成，由 `generator` 模組隨機產生一組符合所有規格條件的週期任務，輸出 `task_set.json`。Phase 2 是日前排程，這是核心：由 `rt_scheduler` 用 PuLP 加 CBC，解一個混合整數線

性規劃，把週期任務展開成具體的 job，在 72 小時上同時最佳化機組成本與市場收益、並滿足全部 23 條限制，輸出 `schedule_result.json`。Phase 3 是接收測試，它已經整合進排程器裡，會在 MILP 解完後自動執行：用 Phase 2 留下的保留量，去接受或拒絕零星的硬截止任務、安排或漏掉非週期的軟截止任務，再把結果標註回每一筆排程紀錄。Phase 4 是效能評估，由 evaluator 讀入排程跟輸入檔，算出所有效能指標，輸出 `evaluation_results.json`。Level 2 在這之上再做兩件事：一是放寬三個 Level 1 的理想假設，二是加上一個會隨著新資訊滾動、不斷重新最佳化的動態排程器。

- 指著架構圖由左到右講資料流：JSON 進、JSON 出，每一階段職責單一。
- 強調 Phase 3 已整合進排程器、自動執行，不是一個獨立的手動步驟。
- 點出 Level 2 = 放寬假設 + 動態排程，自然引到報告後半段。

投影片 3：Phase 1 任務生成（約 2 分鐘）

講稿：Phase 1 的週期任務生成，用的是「生成—算 frame size—驗證」的迴圈：隨機抽一組任務、算出 frame size、丟給驗證器檢查所有規格條件，只要有一條不過就整組重抽，直到全部通過為止；而且我們加了固定的隨機種子，所以這組任務可以完全重現。這裡有一個刻意的設計：為了保證最難的條件一定成立，我們生成的順序是先產生 $d = e$ 的任務，撐住「至少兩成是零鬆弛」這條；再產生 $e = 2$ 的非搶占任務，撐住「至少兩個 $e \neq 1$ 的非搶占任務」這條；其餘的才隨機補足。這次產生的是 8 個週期任務、frame size 等於 4、展開後總共 38 個 job、負載密度 1.31。最重要的是 frame size 的三個條件，這是課本上 frame-based 時鐘驅動排程的經典結果，三條缺一不可。第一條， f 要大於等於最長的執行格數 $\max e_j$ ：這次 $\max e_j = 4$ ，所以 f 至少是 4，意思是一個 frame 要裝得下最長的一段執行，否則某個 job 連在單一 frame 裡都跑不完。第二條，72 要被 f 整除： $72 \bmod 4 = 0$ ，這樣整個 horizon 是 frame 的整數倍，排程表才能整齊地週期重複，尾端不會切出半個 frame。第三條，對每一個任務， $2f - \gcd(f, p_j) \leq d_j$ ：這次最緊的是 p8， $2 \times 4 - \gcd(4, 9) = 8 - 1 = 7$ ，剛好等於 p8 的 $d = 7$ ，通過。這第三條保證的是：在每個 job 從釋放到截止的視窗裡，一定至少存在一個完整、而且可以用的 frame。三條同時成立，所以 f 取最小可行的 4。

- frame size 三條件要逐條唸出實際數字 ($f \geq \max e = 4$ ； $72 \bmod 4 = 0$ ；p8： $8 - 1 = 7 \leq 7$)。
- 主動點出密度 1.31 大於 1 看似奇怪，先解釋或留到被追問。

被追問時：

- 密度為何能 > 1 ？因為一個 processor 可同時供多個 job（規格假設 7），總供電能力大於 1 格/時，這不是單一 CPU 的利用率。
- $d = e$ 的意義？零鬆弛：一釋放就得連續做到截止、沒有任何挪移空間，是最緊迫的硬截止。
- 第三條 $2f - \gcd(f, p) \leq d$ 在保證什麼？保證 job 視窗內至少有一個完整 frame 可

用，是 frame-based 排程可行的充分條件； $\gcd(f, p)$ 是 release 相對 frame 邊界的最差對齊偏移。

投影片 4：Phase 2 變數與線性化（約 2 分鐘）

講稿：Phase 2 的日前排程是一個 MILP，用 PuLP 加 CBC 一次解完整個 72 小時。決策變數有這些： $P_{i,t}$ 是設備 i 在時段 t 的總出力； $k_{j,i,t}$ 是設備 i 在 t 供給需求 j 的能量，如果 j 是充電需求，它就代表灌進電池的量；機組有 u 、start、stop 三個二元變數，表示開關機、啟動、停機；電池有 charge_b、discharge_b 兩個充放電旗標，還有連續的 SOC； $Sell_t$ 是每一格賣到市場的電量； $x_{j,t}$ 是需求 j 在 t 有沒有執行的二元。這裡有一個關鍵理論一定要講：規格的式子大量用 $\min(1, \cdot)$ 來表達「有沒有在作用」這個 0 或 1 的指示，但 $\min$ 是非線性的，MILP 不能直接吃，所以我們的做法是引入二元變數、再用線性不等式把它跟原本的連續量綁定。我舉兩個代表例：第一，job 有沒有執行，我們令 $x = \min(1, \sum_i k)$ ，再用限制式 C1 把它綁成 $\sum_i k = wx$ ，這樣 $x = 1$ 時剛好供滿 w 、 $x = 0$ 時供 0；第二，機組有沒有開機，我們令 $u = \min(1, P)$ ，再用 C6 把它綁成 $outputmin \cdot u \leq P \leq outputmax \cdot u$ ，這樣 $P > 0$ 就逼出 $u = 1$ 、 $u = 0$ 就逼 P 歸零。重點是，這些二元變數都只是既有連續量的「開關指示」，沒有新增任何超出規格語意的自由度——這讓模型在 CBC 下可解，又跟規格的數學式等價。目標函數就是 $\alpha f_1 + f_2 + f_3$ ， $\alpha = 10000$ ， f_1 是非週期逾時數、 f_2 是機組成本、 f_3 是售電收益取負號。

- 強調「一次解完整個 horizon」是靜態法的特徵，與 Level 2 動態法對照。
- 線性化只講兩個代表例（ x 綁 C1、 u 綁 C6），不要把所有二元都列出來。
- 反覆強調「二元只是開關指示、沒有新增決策」——這是 Level 2 規格的紅線。

被迫問時：充放電互斥怎麼線性化？規格 C19 是雙線性 $P \cdot \sum k = 0$ ，我們改用 $charge_b + discharge_b \leq 1$ ，再把功率各自綁到旗標，達到同一格不能又充又放。

投影片 5：23 條限制式（約 2.5 分鐘）

講稿：23 條限制式我們分成七群，剛好對應評分表的 2-1 到 2-7，我不會逐條念，挑骨幹講。第一群是任務與供需路由：C1 規定一個 job 在某一格如果執行，就要當格完整供滿 w ，不能只供一半；C2 是釋放前不能執行；C3 是硬截止的完成性，週期和零星任務在截止前累積執行要恰好等於 e 格，這裡用等號，是要同時排除「沒做完」跟「做超過」；C5 是非搶占任務的連續性，我用「上升沿至多一次」這個二元限制，配合 C3 已經固定總執行格數是 e ，就唯一地逼成單一連續區塊；C20 是每個設備分配出去的總能量不能超過它的出力。第二群 C4 是非週期的軟截止，允許逾時，但要用一個 Miss 變數記下來，逾時就在目標被罰 α 。第三群 C6 到 C12 是傳統機組的物理：出力上下限、爬升率、最小開機跟關機時間、還有跨排程邊界的初始狀態承接。第四群 C13 是再生能源只能用到「額定容量乘上當時段的日照預測比例」這麼多。第五群 C14 到 C21 是儲能，核心是 C16 的 SOC 狀態方程式：本格末的存量等於前一格存量，加上充進來的、減去放

出去的，這其實就是電池的能量守恆，讓電池變成跨時段搬運能量的緩衝器；另外 C19 是同一格不能又充又放、C21 是充電只能由發電設備供應、不能電池互充。最後 C22 是售電非負，因為我們這個模型只賣電不買電；C23 是每小時供需平衡，總發電要剛好等於用電需求加充電消耗加賣到市場的量，這是整個模型的脊椎，把所有設備跟所有需求綁在一起。評分 2-7 是按違反幾小時扣分，所以我們逐格驗證，0 違反。

- 不要逐條念；被問到哪一條，就翻第一部分 §6 的「規格式 + 實作 + 原理」。
- 點名四根骨幹：C3 完成性、C5 連續性、C16 SOC 守恆、C23 供需平衡。
- 補一句：C16/C18 在 Level 2 會被加了效率與自放電的放寬版取代。

被追問時：C5 為何「上升沿至多一次」就等於連續？因為 C3 已經固定總執行格是 e ，再限制狀態從 0 跳到 1 至多發生一次，剩下唯一的可能就是單一連續區塊；把 release 那格也算進可能的上升沿，否則「前一段加後一段」會被漏判。

投影片 6：Phase 3 接收測試（約 2 分鐘）

講稿：Phase 3 是接收測試。MILP 解完之後，每一格沒有賣掉、但其實有發出來的剩餘，就是我們可以挪用的保留量。數學上，每個設備的 spare 等於它的出力 P 減掉已經分配出去的 $\sum k$ ；而由 C23 的平衡，所有設備的 spare 加起來，剛好等於那一格的售電量 Sell。接受一個 job 的時候，我們把它每一格的需求 w 從 spare 轉進 k ，同時把那一格的 Sell 等額減少。關鍵就在這裡：因為「轉進去的量」跟「少賣的量」相等，C23 的兩邊同時減掉同一個數，平衡還是成立；而且我們只動用了 spare、也就是 $P - \sum k \geq 0$ 的那部分，所以 C20 設備上限也不會被破壞。也就是說，接收測試完全不需要重解 MILP，只是在既有的最佳解的縫隙裡塞工作。判斷規則上：如果是可搶占的 job，就在視窗內挑出所有剩餘保留大於等於 w 的時槽，數量夠 e 個就取最早的 e 個；如果是非搶占，就要找一段長度 e 的連續視窗，而且每一格的剩餘都要大於等於 w 。零星是硬截止，找不到合格時槽就直接拒絕；非週期是軟截止，盡量排到 72 小時前完成，完成晚於軟截止就記一個 miss。零星任務的價值，我們用 value rate 衡量，它是「在硬截止前實際完成的執行格總長」除以「全部零星的執行格總長」，而不是用「接受了幾個」——因為接受了卻沒做完，是沒有意義的。這次 6 個零星全部接受、而且準時完成，value rate 等於 1.0，是滿分段；非週期在靜態下有 a2 跟 a5 兩個逾時，所以靜態的軟逾時率是 0.2。

- 講清楚「轉移剩餘、扣售電、平衡不變、不重解」這個核心手法 (§7)。
- value rate 是「準時完成的執行格佔比」，不是「接受幾個」。
- 每一筆接受/拒絕都有理由寫進 acceptance_test_log.json，可以被質詢。

被追問時：非搶占 job 被拒絕的理由長什麼樣？例如「[28, 33] 內找不到連續 2 格皆 $\geq w$ 的保留」，每一筆都逐 job 記錄在 log 裡。

投影片 7：Level 2-A 放寬三假設（約 2.5 分鐘）

講稿：Level 2 的第一部分是放寬假設。Level 1 有幾個理想化的假設，我們放寬其中三個：假設 11，再生能源的預測一定準；假設 12，理想儲能；假設 5，工作之間沒有優先序。這裡有一條硬規定：規格允許我們修改符號、增加參數，但不能新增任何決策變數，所以底下全部十條，都只用 Level 1 原本就有的 $P, k, SOC, Sell, x$ 。先講再生能源：R1 把日前用的預測折減 15%，這是穩健最佳化的保守邊際——因為預測常常高估，如果押滿預測、實際短缺就會缺電，所以寧可少規劃一點再生能源、靠可控的機組去補位。儲能的部分最豐富：R2 到 R4 把 SOC 平衡式加上充電效率 0.95、放電效率 0.95、還有每一格的自放電，把理想電池變成往返效率大約 0.9 的真實電池，套利的價差要大於損耗才划算；R5 把放電量的上限也扣掉效率跟自放電；R6 是循環吞吐上限，全期的總放電量不超過 3 個滿可用容量，等於全期最多等效 3 次完整循環，保護電池壽命；R7、R8 是 SOC 相依的放電跟充電功率，模擬電池快空或快滿的時候功率會下降，也就是鋰電池的 CC-CV 行為；R9 把老化成本放進目標，每放電 1 MWh 罰 2 元，讓最佳化在「操電池」跟「多燒一點燃料」之間權衡。最後優先序 R10，用一個有序對 (a, b) 表示 a 全部做完之前 b 不能開始，這就是任務間的前置依賴，例如「先抽水才能曝氣」，它重用 x 變數、沒有新變數。這樣總共 R1 到 R10 十條放寬限制，每一條對應評分 1 分、上限 10 分；而且只要把所有參數設回預設值，模型就會跟 Level 1 完全一樣——這是「一條放寬得 1 分」能夠逐條被驗證的前提。

- 反覆強調「可改符號、可加參數，但不新增決策變數」，這是這項評分的關鍵。
- 舉兩個代表：R2–R4（SOC 平衡加效率與自放電）、R9（老化成本進目標）。
- 強調預設參數（ $\eta = 1, \sigma = 0, \beta = 0, N = \infty, \varphi = 1, c_{age} = 0$ ）就還原成 Level 1，可逐條驗證。

被追問時：怎麼證明沒有新增決策變數？R1–R10 全部只重用 $P, k, SOC, Sell, x$ ，沒有宣告新的變數；而且 `validator --level 2` 會用放寬後的限制式逐條重驗。

投影片 8：Level 2-B 動態排程方法（約 2 分鐘）

講稿：Level 2 的第二部分是動態排程，方法是滾動視窗再最佳化，也就是模型預測控制 MPC。對照一下：Level 1 是在 $t = 0$ 一次解完整個 72 小時；動態法則是沿著時間往前走，在幾個觸發點上，對「還沒執行的剩餘時程」重新解一次 MILP，用來因應靜態計畫看不到的動態資訊，像是太陽能的實際出力、還有臨時任務的到達。我們選 MPC，而不是完全重排或啟發式，是因為它同時兼顧最佳性跟可解性：每一次都解一個真正的 MILP，所以是最佳、而且滿足全部限制；但只解尾段，規模隨著時間縮小，所以線上秒級就能解出來。核心機制有三個。第一是狀態承接，我們叫 `pin_prefix`：每次觸發時，把已經執行的前段釘成已承諾的值，包括連續變數 $P, k, SOC, Sell$ ，再加上由它們推導出來的前綴二元 u 、`charge_b`、`discharge_b`、 x ，只重解後面那一段。釘住連續狀態，機組的開關機時長、爬升位置、SOC 就能正確跨界承接；再固定前綴二元，

solver 的 presolve 就能把凍結的那一段整段消去，所以每次再解都很便宜。第二是觸發時機：固定每 24 格一次的週期節奏、每一個零星任務一釋放就觸發（因為硬任務要即時審核）、還有最多 3 個「實際太陽能跟預測偏差最大」的時點。第三是線上接收等於保留可行性：任務在釋放時揭露，能不能被接受，取決於再最佳化能不能在它的執行視窗內保留足夠的可挪用剩餘，也就是讓 $Sell \geq R_t$ ；解得出來就接受，解不出來就丟掉最低優先的任務再重解，硬任務不可行就拒絕、軟任務記 miss。被接受的任務會隨著區塊凍結被路由進排程，所以接受的硬任務一定在截止前完成。每一次再解，我們設了 MIP gap 0.20、時間上限 20 秒、單執行緒，確保線上可解、又可重現。

- 先講對照基準：L1 一次解完 vs L2 沿時間滾動、只重解尾段。
- 「釘前綴 + 只重解尾段 + presolve 消去」是線上可解的關鍵。
- 三類觸發：節奏 24 格 / 零星到達 / PV 偏差最大點；gap 0.20、時限 20s、單執行緒、固定種子 ⇒ 可重現。

被追問時：

- 再生能源的「實際值」哪裡來？用種子化隨機模型在預測周圍加噪音 (std 0.2、seed 42) 生成，已承諾區塊用實際值、未見尾段用折減預測，對過去精確、對未來保守。
- 釘前綴會不會讓結果變差？釘住的是已經實際執行、不可改的過去；尾段仍是完整 MILP 最佳解，不影響可達到的最佳性。

投影片 9：結果 正確性 (約 1.5 分鐘)

講稿：先看正確性。動態法這次總共跑了 9 輪再最佳化、解了 9 次 MILP，單執行緒、固定種子，可以完整重現。任務是隨著時間揭露、再被接受的：第 2 小時揭露 s1、a1、a2，第 14 小時是 s2、a3，第 25 小時是 a4，一路到第 66 小時揭露 s6、a10；每一輪都全部接受、沒有任何拒絕、也沒有任何再排程失敗。最後的結果是：38 個週期 job、6 個零星 job 全部在硬截止前完成，10 個非週期 job 也全部在軟截止前完成。我們用放寬後的限制式逐格重新驗證：C23 供需平衡 0 違反、SOC 全程維持在上下限之內 0 違反、放寬後的儲能、機組、再生能源限制每一輪都成立。也就是說整個動態過程，沒有漏掉任何一個硬截止、沒有任何 miss、也沒有重排失敗。我們是用 validator --level 2 去逐項重驗的，驗的是「實作」、不只是「描述」。

- 強調 9 輪 / 9 解 / 單執行緒 / 固定種子可重現。
- 用揭露表唸出 $2 \rightarrow 14 \rightarrow 25 \rightarrow \dots \rightarrow 66$ 的任務到達順序。
- 強調逐格驗證 C23、SOC 都 0 違反，是用程式重驗，不是口頭宣稱。

投影片 10：結果 靜態 vs 動態 (約 2 分鐘)

講稿：這是最關鍵的比較，靜態 Level 1 對動態 Level 2。靜態日前計畫雖然一次就看到所有任務，但它只承諾一份排程、保留量有限，結果漏掉 2 個非週期任務 a2 跟 a5，

軟逾時率 0.2。動態法隨著任務到達再最佳化、而且主動為它們保留容量，10 個非週期全部準時，軟逾時率降到 0，平均回應時間也從 3.63 降到 2.65。我把這筆帳算給老師聽：消除這 2 個軟逾時，等於省下 2 倍的 α 、也就是 2 萬元的罰則，這主導了總目標的改善，目標值從 -69 410 改善到 -83 239。代價也很清楚：為了保留容量、又要補償太陽能實際的短缺，機組成本上升了大約 1 萬 5；但同時因為承諾發更多電，售電收益也增加了大約 8 千 9。所以即使多花了成本，扣掉省下的逾時罰則之後，動態法在總目標上明顯更好。這正好印證了我們前面講的權衡：提升接收率，必須用成本或收益來換，三個目標沒辦法同時最佳，這是加權和之下的 Pareto 取捨。

- 把「省 2 萬罰則 > 多花 1.5 萬成本、而且還多賺 0.9 萬收益」這筆帳明確算出來。
- objective_value 從 -69 410 到 -83 239 是主導指標。
- 收束到權衡論述（評分 6-2）：接收率上升要用成本/收益換取。

被追問時：jitter 動態略增 (44.1 → 45.8) 為何仍較好？jitter 是同一週期任務各 instance 完成時刻的離散度；動態為了保留容量微調了部分週期 job 的完成時刻，但硬截止全守住、總目標更佳，是可接受的取捨。

投影片 11：結語與 AI 協作（約 1 分鐘）

講稿：最後做個總結。Level 1 我們完成了任務生成、日前 MILP 排程、接收測試、效能評估四個階段，23 條限制逐條對應到規格的數學式跟設計原理；Level 2 我們放寬了再生能源不確定、真實儲能、工作優先序三個假設、總共十條放寬限制，並且做了一個滾動視窗的動態排程器，用實際數據證明動態法能在完全不漏掉任何硬截止的前提下，把軟逾時率降到 0、總目標也更佳。整個程式採物件導向、單一職責的設計，每一個類別只負責一件事，所有結果都用固定種子可以重現。關於 AI 協作的部分：我們用 AI 輔助建模的思路跟程式骨架，但模型的正确性驗證、參數的調校、還有 23 條限制式逐條的交叉檢查，都是我們自己負責完成的。報告到這裡，謝謝老師。

- 一句話收束 Level 1 加 Level 2 的成果。
- 主打結論：不漏硬截止、軟逾時 0、總目標更佳、可重現。
- AI 協作誠實交代分工：AI 輔助建模與骨架，人負責驗證、調參、交叉檢查。

預期問答（備用）

- **Q：為何用 MILP 而非啟發式？** A：MILP 在 23 條硬限制下保證可行且最佳；線上再最佳化用 MIP gap 與時間上限即可秒級解出，兼顧最佳性與可解性。啟發式雖快但無法保證滿足全部硬限制、也難證明最佳。
- **Q：為何 $\alpha = 10000$ ？** A：它是把「漏任務個數」折算成金額的匯率，且刻意遠大於單一任務在不同時段供電的邊際成本差（數百到一千元量級），使最佳化近似字典序：先盡力不漏任務、在此前提下才壓成本、衝收益——這正是「先保證時限、再談效率」的即時系統精神。

- **Q：怎麼確定沒有新增決策變數？** A：R1–R10 與保留下限 $Sell_t \geq R_t$ 都只重用既有的 $P, k, SOC, Sell, x$ ，沒有宣告新變數；且 `validator --level 2` 會用放寬後的限制式逐條重驗。
- **Q：接收測試為何不用重解 MILP？** A：接受一個 job 時把需求從 `spare` 轉進 k 、同時把 `Sell` 等額扣掉，C23 兩邊同減一個數、平衡不變；又只動用 $P - \sum k \geq 0$ 的縫隙，C20 設備上限也不破壞，所以是在既有最佳解上塞工作，不需重解。
- **Q：pin_prefix 釘住前綴會不會讓結果變差？** A：釘住的是已經實際執行、不可更改的過去；只額外固定前綴二元讓 `presolve` 加速，尾段仍是完整 MILP 的最佳解，不影響「從現在起可達到的最佳性」。
- **Q：每次再解 gap 0.20 會不會不夠最佳？** A：gap 是「離最佳解的相對上界 20%」，用來換取秒級可解；實測 9 輪全部接受、無 miss，且總目標明顯優於靜態，足以支撐結論。若要更緊可調小 gap、放寬時限。
- **Q：再生能源的「實際值」怎麼產生？** A：用種子化隨機模型在預測周圍加噪音 (noise std 0.2、seed 42) 生成「實際」PV；已承諾區塊用實際值、未見尾段用折減後的預測 (R1)，對過去精確、對未來保守。
- **Q：frame size 那條 $2f - \gcd(f, p) \leq d$ 在保證什麼？** A：保證每個 job 的視窗 $[r, r + d - 1]$ 內至少存在一個完整、可用的 frame，是 frame-based 排程可行的充分條件； $\gcd(f, p)$ 是 release 相對 frame 邊界的最差對齊偏移。
- **Q：負載密度 1.31 為何能大於 1？** A：因為一個 processor 可同時供應多個 job (規格假設 7)，總供電能力大於 1 格/時，這不是單一 CPU 的利用率上限。
- **Q：Level 2 放寬之後 Level 1 還跑得動嗎？** A：能。所有放寬參數預設為 `no-op` ($\eta = 1, \sigma = 0, \beta = 0, N = \infty, \varphi = 1, c_{age} = 0$)，此時模型與 Level 1 完全相同。
- **Q：jitter 動態略增 (44.1 → 45.8) 為何仍較好？** A：jitter 是同一週期任務各 instance 完成時刻的離散度；動態為保留容量微調了部分週期 job 的完成時刻，但硬截止全守住、軟逾時降到 0、總目標更佳，屬可接受取舍。

(報告結束)